

Edu-Ransomware

Architecture, Features, and Implementation Details

IT Security – Educational Ransomware Simulation Project

March 1, 2026

Important Notice

This software was developed **exclusively for educational purposes** and security research.

- Using this software on systems without explicit authorization is **illegal**.
- **NEVER** run this malware on production systems.
- Always use an isolated Virtual Machine (VM).
- The Pinggy tunnel links expire after 60 minutes as a safety mechanism.

Repository: <https://github.com/TimBLukas/rust-mw>

License: MIT (with Ethical Use Clause)

Agenda

- Project Overview
- System Architecture
- Attack Kill Chain
- Delivery Mechanisms
- Evasion Module
- Network Communication
- Encryption Engine
- Persistence Mechanisms
- Data Exfiltration
- Extortion Features
- Network Reconnaissance
- Safety and Stealth Features
- Stealth Execution
- Testing Strategy
- MITRE ATT&CK Mapping

Project Overview

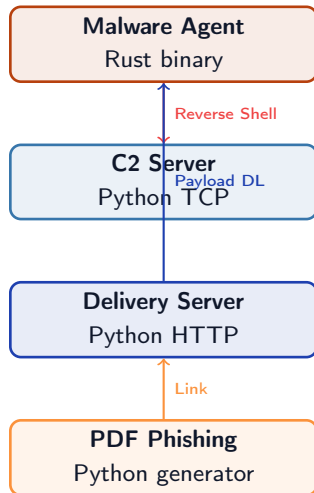
What is Edu-Ransomware?

A full-stack ransomware simulation built for educational purposes to demonstrate how modern ransomware attacks work end-to-end.

Components:

1. **Rust** Malware Agent – Executes on victim system
2. **Python** C2 Server – Command and control
3. **Python** Delivery Server – Drive-by-download
4. **Python** PDF Phishing – Social engineering

Goal: Investigate how complex it is for beginners to create functional ransomware with modern tools and AI assistance.



Technology Stack

Malware Agent –

Why Rust?

- Compiles to native static binary (no runtime needed)
- Cross-compilation: Windows, Linux, macOS from one codebase
- Memory safety without garbage collector
- Complex binaries are harder to reverse-engineer
- Growing trend in real-world malware (BlackCat/ALPHV)

Key Crates:

- `aes`, `ctr` – AES-256-CTR encryption
- `walkdir` – Recursive filesystem traversal

C2 Server – Python

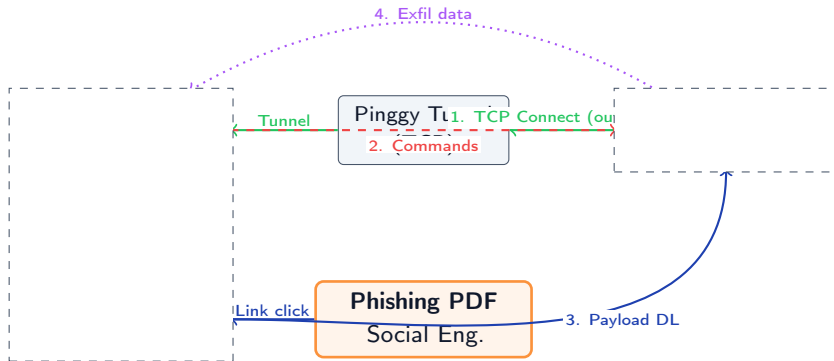
- Multithreaded TCP socket server
- Interactive CLI for operator control
- Base64 stream decoding for exfiltration
- Session management for multiple victims
- ANSI-colored terminal output

Infrastructure

- **Tunneling:** Pinggy.io TCP tunnels (NAT traversal)
- **Delivery:** Python HTTP server with OS detection
- **Build:** Cargo cross-compilation + shell scripts

System Architecture

High-Level Architecture



Project Structure

rust-mw/

└─ malware_agent/ (Rust)

└─ src/

└─ main.rs

└─ lib.rs

└─ config.rs

└─ crypto/ (AES, Hybrid)

└─ evasion/ (HW, Behavior, Process)

└─ network/ (C2, DGA, Discovery, Exfil)

└─ recon/ (LDAP, SMB)

└─ stealth/ (Antiforensics, Mutex)

└─ safety/ (Killswitch, Geofencing)

rust-mw/ (continued)

└─ c2_server/ (Python)

└─ server.py

└─ loot/

└─ delivery/ (Python)

└─ DbD-Site/

└─ server.py

└─ public/ (HTML templates)

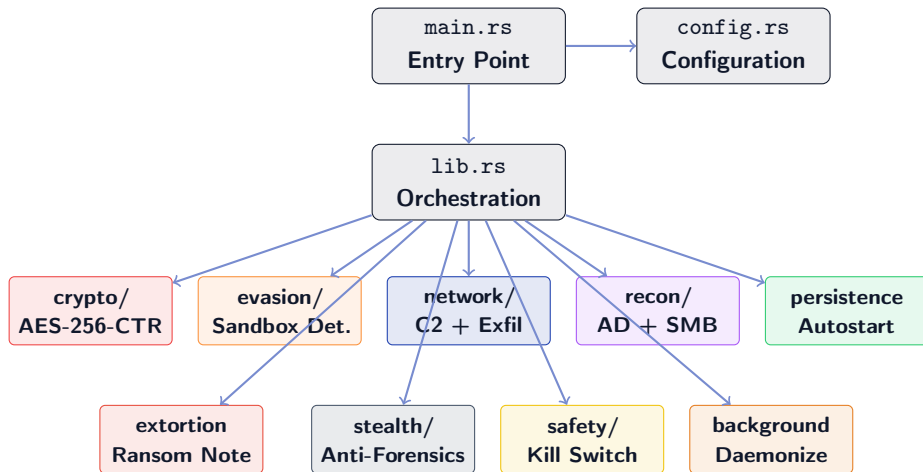
└─ files/ (Payloads)

└─ pdf_phishing/

└─ scripts/

└─ Dockerfile

Modular Agent Architecture



Attack Kill Chain

Attack Flow – Code Walkthrough

Execution sequence of the Rust agent (`main.rs`):

1. **Daemonize** (`background.rs`)
 - Linux: `fork()` to background; Windows: `windows_subsystem` attribute
2. **Sandbox Detection** (`evasion/`)
 - **Pass:** Environment appears safe → continue
 - **Fail:** Fake error message → **exit immediately**
3. **Establish Persistence** (`persistence.rs`)
 - Set up autostart to survive reboots
4. **C2 Loop** (`network/c2.rs`)
 - Connect to C2 server, await commands
5. **On Command: Encrypt** (`crypto/aes.rs`)
 - AES-256-CTR encryption of target directory
6. **On Command: Extort** (`extortion.rs`)
 - Deploy ransom note, change wallpaper

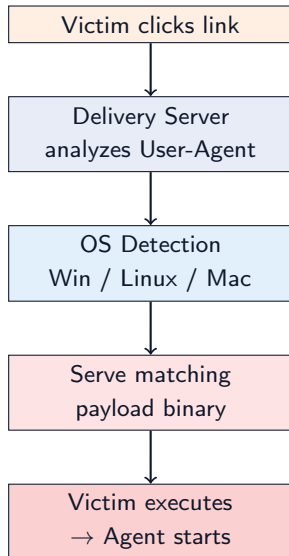
Delivery Mechanisms

Drive-by Download

Smart payload delivery based on OS detection:

The delivery server (`delivery/DbD-Site/server.py`) provides:

- Multiple phishing page templates:
 - `/game` – Fake gaming download
 - `/security` – Fake security alert
 - `/prize` – Fake prize notification
- `/get_document` – Smart download endpoint
- **User-Agent detection:** Identifies Windows, Linux, or macOS
- Delivers OS-specific payload automatically



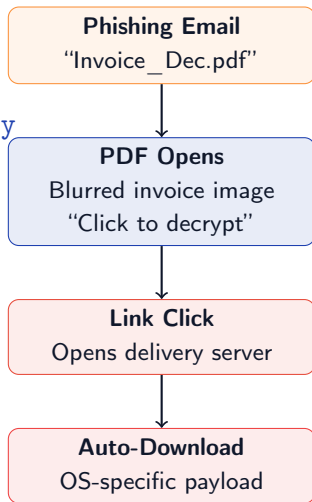
PDF Phishing

Social engineering via crafted PDF:

- Generated by `delivery/pdf_phishing/generate_phishing_pdf.py`
- Contains a **blurred invoice image** as bait
- Button labeled “Decrypt Content” (social engineering)
- Clicking the button opens the delivery server URL
- Delivery server then serves the OS-specific malware

Attack scenario:

1. Employee receives email with



Evasion Module

Multi-Layer Sandbox Detection

Module: `evasion/` – Checks run sequentially (fast checks first):

1. Hardware Checks

`hardware.rs`

- RAM < 3 GB? → VM/Sandbox
- CPU cores < 3? → VM
- Disk space < 60 GB? → VM

Uses `sysinfo` crate for cross-platform system queries.

2. Process Scan

`process.rs`

Checks for analysis tools:

- Wireshark
- Process Monitor
- x64dbg / OllyDbg
- Fiddler
- IDA Pro

If found: immediate exit.

3. Behavior Check

`behavior.rs`

- Record mouse position
- Sleep for 5 seconds
- Record mouse position again
- If no movement: **sandbox!**
- Also detects sleep acceleration

Run last (slowest check).

Network Communication

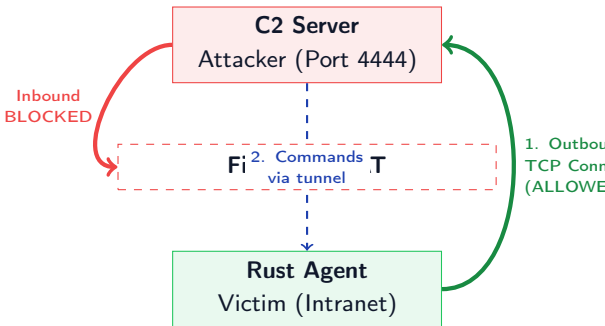
Reverse Shell Architecture

Why reverse shell?

- Victim connects **outward** to attacker
- Bypasses firewall rules (outbound traffic typically allowed)
- NAT traversal via Pinggy.io TCP tunnel

Connection Flow:

1. Agent reads C2 address from `config.rs`
2. Connects via `TcpStream::connect()`
3. Enters command loop: receive → execute → respond



C2 Command Set

Commands processed by `network/c2.rs` → `process_command_logic()`:

Command	Description	Module
<code>shell <cmd></code>	Execute OS command on victim	<code>c2.rs</code>
<code>encrypt [path]</code>	Encrypt target directory (AES-256-CTR)	<code>crypto/aes.rs</code>
<code>decrypt [path]</code>	Decrypt <code>.locked</code> files using <code>rescue.key</code>	<code>crypto/aes.rs</code>
<code>exfil <file></code>	Exfiltrate single file (Base64-encoded)	<code>c2.rs</code>
<code>auto-exfil</code>	Auto-exfiltrate high-value files (background)	<code>exfiltration.rs</code>
<code>exfil-screenshot</code>	Capture and exfiltrate screenshot	<code>exfiltration.rs</code>
<code>recon</code>	AD/LDAP + SMB network reconnaissance	<code>recon/</code>
<code>kill</code>	Self-destruct: delete traces + exit	<code>stealth/</code>

Fallback Behavior

Any unrecognized command is passed to the OS shell (`sh -c` on Linux, `cmd /C` on

C2 Server CLI

`sessions` – List connected victims

`interact <id>` – Interactive shell

Encryption Engine

AES-256-CTR Atomic Encryption

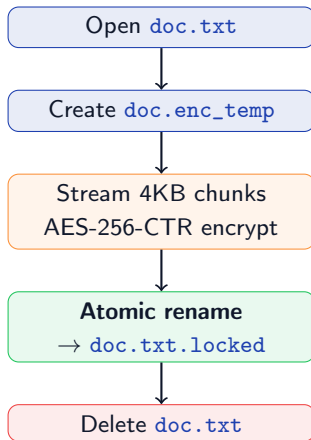
Module: `crypto/aes.rs`

Algorithm: AES-256 in CTR (Counter) mode

- Stream cipher – processes data in chunks (4 KB)
- Symmetric key: 32 bytes (256 bits), randomly generated
- IV: 16 bytes (currently static for simplicity)
- Key saved to `rescue.key` in target directory

Atomic file operation:

1. Open original file for reading



Cras
tem
origi
exist

Encryption Orchestration

Module: `lib.rs` → `run_malware_logic()`

Full encryption workflow:

1. Validate target directory exists
2. Create and show ransom note (`README_DECRYPT.html`)
3. Generate random AES-256 key
4. Save key to `rescue.key`
5. Walk filesystem recursively (`walkdir`)
6. For each file:
 - Skip: `rescue.key`, `README_DECRYPT.html`, `.locked`, `.enc_temp`
 - Encrypt with `encrypt_file_atomic()`
7. Change desktop wallpaper to ransom image

Decryption

(`run_decryption_logic`):

1. Read `rescue.key` from target dir
2. Validate key length (32 bytes)
3. Walk filesystem for `.locked` files
4. Decrypt each with same AES-CTR key
5. Restore original filename
6. Delete `.locked` file

Persistence Mechanisms

Cross-Platform Persistence

Module: `persistence.rs` – Survives system reboots on all platforms:

Linux (Dual Method)

Method A: Systemd User Service

- Creates `.service` file in `~/.config/systemd/user/`
- Type: `forking`
- Enabled via `systemctl -user`
- No root required

Method B: Cron Job

- Adds `@reboot` entry

Windows (Dual Method)

Method A: Scheduled Task

- Uses `schtasks.exe`
- Trigger: `onlogon`
- Name: `WindowsUpdateHelper`
- Requests highest privileges
- `CREATE_NO_WINDOW` flag

Method B: Registry Run Key

- `HKCU\...\Run`

macOS

LaunchAgent

- Creates `.plist` in `~/Library/LaunchAgents/`
- Label: `com.service.updater`
- `RunAtLoad: true`
- No root needed

Design Pattern

Uses `cfg!` compile-time
attributes

Data Exfiltration

Exfiltration Capabilities

Module: `network/exfiltration.rs` – Double extortion support:

Manual Exfiltration

- C2 command: `exfil <filepath>`
- File read → Base64 encode → send
- Format: `EXFIL_DATA:<filename>:<base64>`
- Server decodes and saves to `loot/` directory

Automated High-Value Exfiltration

Command: `auto-exfil`

- Runs in background thread (non-blocking)
- Searches for high-value files by keyword:
- `password`, `secret`, `iban`, `credit`

Screenshot Capture

Command: `exfil-screenshot`

- Captures screen using `screenshots` crate
- Encodes as PNG → Base64
- Sent via dedicated TCP connection
- Saved as image file in `loot/`

Server-Side Handling

The C2 server (`server.py`) uses a 1 MB receive buffer and accumulates chunks until transfer completes. Handles `EXFIL_DATA`

Extortion Features

Visual Impact – Ransom Note and Wallpaper

Module: `extortion.rs` – Maximum psychological pressure:

Ransom Note (HTML)

- File: `README_DECRYPT.html`
- Full HTML/CSS with dark theme
- Red/black color scheme for urgency
- “YOUR FILES ARE ENCRYPTED” heading
- Instructions to contact C2 operator
- “Decrypt Files Now” button (opens alert)
- Auto-opened in default browser:
 - Linux: `xdg-open`
 - Windows: `cmd /C start`
 - macOS: `open`

Desktop Wallpaper Change

- Embedded image: `ransom_wallpaper.jpg`
- Included at compile time via `include_bytes!`
- Extracted to filesystem at runtime
- Set using `wallpaper` crate (cross-platform)
- Requires absolute path (`fs::canonicalize`)
- Mode: Crop (fills entire screen)

Psychological Effect

The combination of encrypted files, browser popup, and changed wallpaper creates imme-

Network Reconnaissance

Active Directory and SMB Reconnaissance

Module: [recon/](#) – Enterprise environment enumeration:

LDAP Enumeration ([ldap.rs](#))

- Auto-detects Domain Controller:
 - Windows: [LOGONSERVER](#) env var
 - Linux: [krb5.conf](#) / DNS SRV records
- Enumerates without admin rights:
 - Domain Admins group members
 - Backup Operators
 - All computer objects (FQDN)
 - Domain Controller identification
- Uses [ldap3](#) Rust crate

SMB Share Enumeration ([smb.rs](#))

- Cross-platform:
 - Windows: [net view](#)
 - Linux: [smbclient](#)
- Port 445 connectivity check
- Read/write access testing
- Filters for encryption targets (writable disk shares)
- Subnet scanning for SMB hosts ([/24](#))

High-Value Target ID ([mod.rs](#))

Auto-detects critical systems by hostname

Safety and Stealth Features

Safety Mechanisms

Built-in safeguards to prevent accidental harm:

Kill Switch ([safety/killswitch.rs](#))

- HTTP check to remote URL before execution
- Response “STOP” → agent exits and self-deletes
- Response “RUN” or 404 → continue
- Configurable via [config.rs](#)
- Allows remote emergency shutdown

Geo-Fencing ([safety/geofencing.rs](#))

- Checks local IP address
- Only allows private networks: [192.168.x.x](#),
[10.x.x.x](#)

Singleton / Mutex ([stealth/mutex.rs](#))

- Named lock prevents multiple instances
- Uses [named_lock](#) crate
- If lock already held → exit silently
- Prevents double-encryption scenarios

Anti-Forensics

([stealth/antiforensics.rs](#))

- [kill](#) command triggers full cleanup:
 - Delete the executable itself
 - Remove persistence entries
 - Wipe shell history

Configuration System

Module: `config.rs` – Centralized compile-time configuration:

Config Structure

```
pub struct Config {  
    pub ip_address: &'static str,  
    pub port: u16,  
    pub enable_evasion: bool,  
    pub enable_killswitch: bool,  
    pub safe_mode: bool,  
}  
  
impl Config {  
    pub const fn default() → Self {  
        Config {  
            ip_address: CONFIG_IP_ADDR,  
            port: CONFIG_PORT,  

```

Configuration Fields:

Field	Purpose
<code>ip_address</code>	C2 server hostname
<code>port</code>	C2 server port
<code>enable_evasion</code>	Toggle sandbox checks
<code>enable_killswitch</code>	Toggle kill switch
<code>safe_mode</code>	Educational watermark

Compile-Time Config

Uses `const fn` so the configuration is embedded at compile time. The Pinggy tunnel address must be updated before each build.

Stealth Execution

Background Execution and Window Hiding

Module: `background.rs` – Making the agent invisible:

Linux: Process Daemonization

- Uses `fork()` system call
- Parent process exits immediately
- Child continues in background
- Detached from terminal session
- No visible window or process name clue
- Standard daemon pattern

Windows: Subsystem Trick

- Compile-time attribute:
`#![windows_subsystem = "windows"]`
- Tells Windows: “this is a GUI app”
- No console window is created
- Process runs silently in background
- Invisible in taskbar
- Only visible in Task Manager

Debug vs Release

The `windows_subsystem` attribute is only active in release builds

Testing Strategy

Comprehensive Test Suite

Every module includes unit tests (`#[cfg(test)]` blocks):

Crypto Tests

- Key generation (32-byte length, file persistence)
- Full encrypt/decrypt cycle (data integrity)
- Non-locked files are ignored during decryption
- Temp directory isolation for each test

Network Tests

- Shell command execution (echo, error handling)
- Command parsing: encrypt, decrypt, exfil
- Custom path handling with trim

Extortion Tests

- HTML content integrity checks
- File creation and path validation
- Content match: written vs. expected
- Permission error handling (invalid paths)
- Wallpaper function with missing files

Persistence Tests

- Service file content generation
- Systemd file creation in temp dirs
- Cron entry string formatting

MITRE ATT&CK Mapping

Tactics and Techniques Coverage

Tactic	Technique	ID	Module
Initial Access	Drive-by Compromise	T1189	delivery/
Initial Access	Phishing: Spearphishing Link	T1566.002	pdf_phishing/
Execution	Command & Scripting Interpreter	T1059	network/c2.rs
Persistence	Boot/Logon Autostart	T1547.001	persistence.rs
Persistence	Scheduled Task/Job	T1053.005	persistence.rs
Defense Evasion	Virtualization/Sandbox Evasion	T1497	evasion/
Defense Evasion	Indicator Removal	T1070	stealth/
Discovery	System Information Discovery	T1082	evasion/hardware.rs
Discovery	Domain Account Discovery	T1087.002	recon/ldap.rs
Discovery	Network Share Discovery	T1135	recon/smb.rs
Collection	Screen Capture	T1113	exfiltration.rs
Exfiltration	Exfil Over C2 Channel	T1041	exfiltration.rs
Impact	Data Encrypted for Impact	T1486	crypto/aes.rs
Impact	Defacement: Internal	T1491.001	extortion.rs

Conclusion

Key Findings

Low Barrier to Entry

Starting from zero Rust experience, a fully functional ransomware was developed. Modern tooling and AI assistance make the barrier to entry **alarmingly low**.

AV Evasion

The Rust-compiled agent **was not detected** by common antivirus solutions during testing. Rust's complex binary output and lack of signatures contribute to evasion.

Defense Gaps

Static analysis alone is insufficient. Behavioral detection, network monitoring, and proper backup strategies remain critical defense layers.

Conclusion

The goal of this project was to investigate how complex ransomware development is for beginners. The result: **with modern tools and AI assistance, the barrier is disturbingly low**. This underscores the critical importance of proactive cybersecurity measures and continuous security awareness training.

Future Work

Planned Enhancements:

- **Hybrid Encryption:** RSA key wrapping for AES keys (no local key recovery)
- **Intermittent Encryption:** Partial file encryption for speed
- **Domain Generation Algorithm:** Resilient C2 fallback
- **Network Discovery:** Ping sweep and lateral movement simulation
- **Improved Ransom Note:** JavaScript countdown timer, chat simulation

Blue Team Features:

- **YARA Rule Generator:** Auto-generate detection signatures
- **IOC Extractor:** Export indicators of compromise report
- **Safe-Mode Watermark:** Educational marker in encrypted files
- **DevContainer:** One-click lab setup with VS Code
- **Full MITRE ATT&CK documentation**

- [1] IT Security Lecture, Prof. Dr. Matthias Hamann, WS24/25.
- [2] Razulla, B. et al.: *The Age of Ransomware: A Survey on the Evolution, Taxonomy, and Research Directions*.
- [3] ENISA (2024): *Threat Landscape Report 2024*.
- [4] *Edu-Ransomware Repository* (GitHub, 2026):
<https://github.com/TimBLukas/rust-mw>
- [5] Morato, D. et al.: *Ransomware early detection by the analysis of file sharing traffic*.
- [6] MITRE ATT&CK Framework: <https://attack.mitre.org>